

ANTEPROYECTO DEL TRABAJO DE FIN DE GRADO

Alumno/a	Eduardo González Bautista				
Titulación:	Grado en Ingeniería del Software				
Tutor/es:	Gabriel Jesús Luque Polo Zakaria Abdelmoiz Dahi				
Título	Transpilación Cuántica Híbrida: Optimización de Layout Multiobjetivo y Síntesis mediante Aprendizaje por Refuerzo.				
Subtítulo <i>(solo si en grupo)</i>					
Título en inglés	Hybrid Quantum Transpilation: Multi-Objective Layout Optimization and Reinforcement Learning Synthesis.				
Subtítulo en inglés <i>(solo si en grupo)</i>					
Trabajo en grupo:	☒ Sí	☐	☐ No	☒ X	
Otros integrantes del grupo:					

Contextualización del problema a resolver. Describir claramente de dónde surge la necesidad de este TFG y el dominio de aplicación. En caso de que el TFG se base en trabajos previos, debe aclararse cuáles son las aportaciones del TFG.

La computación cuántica está basada en conceptos de la mecánica cuántica, como por ejemplo el entrelazamiento, que le permite alcanzar una aceleración del cómputo que resulta bastante interesante por varias aplicaciones [1].

En la actualidad existen dos paradigmas de computación cuántica principales existen: basado en puertas y el basado en enfriamiento cuántico. El primero paradigma siendo más amplio y permite una multitud de aplicaciones tal como la resolución de problemas de optimización y la simulación de sistemas cuánticos, mientras el segundo está dedicado solamente a la resolución de problemas de optimización. La forma de programar los sistemas del primer paradigma es el desarrollo de un circuito cuántico que realice el algoritmo deseado.

La transpilación de circuitos cuánticos es un proceso crítico para adaptar algoritmos abstractos a las restricciones físicas del hardware real (conectividad, conjunto de puertas nativas, etc.). Este proceso de transpilación [4] es bastante complejo e incluye un número importante de transformaciones o fases: mapear los cúbits lógicos a físicos, transformar las puertas utilizadas en el circuito por las puertas admitidas por el equipo real, añadir nuevas puertas para conseguir la comunicación entre cúbit que deben actuar en conjunto, etc. Para esto, actualmente, enfoques punteros como el de IBM utilizan Aprendizaje por Refuerzo (o *Reinforcement Learning*, RL) para la síntesis de circuitos [2], pero dependen de heurísticas estáticas y a menudo aleatorias (como SABRE) para la fase inicial de *layout* (mapeo inicial de cúbits) [4]. Esta inicialización subóptima limita la eficiencia que el agente de RL puede alcanzar posteriormente y del resultado final del proceso de transpilación. Además, la calidad de dicha inicialización se enfoca únicamente en una sola métrica que refleja una sola calidad del cómputo (i.e. circuito) resultante.

Este Trabajo de Fin de Grado propone un enfoque híbrido que sustituye dicha inicialización heurística por una fase de Optimización Multiobjetivo (MO) que considera varios aspectos de calidad de la transpilación simultáneamente [3]. El objetivo es generar *layouts* iniciales de alta calidad de cara a la eficiencia del cómputo y su resiliencia al error que maximicen el rendimiento de la posterior fase de síntesis basada en RL. Las aportaciones principales serán el diseño de este pipeline híbrido y la evaluación de si una mejor inicialización permite obtener circuitos finales superiores en términos de calidad y ejecución.

Descripción detallada de en qué consistirá el TFG. En caso de que el objeto principal del TFG sea el desarrollo de software, además de los objetivos generales deben describirse sus funcionalidades a alto nivel.

El objetivo principal de este TFG es investigar, diseñar e implementar una arquitectura de transpilación híbrida que combine optimización multiobjetivo y aprendizaje por refuerzo. El propósito es superar las limitaciones de las heurísticas de inicialización actuales (como SABRE) mediante la generación de *layouts* optimizados que maximicen el rendimiento de la posterior síntesis del circuito cuántico.

El desarrollo de software se realizará de manera incremental, abordando los siguientes objetivos específicos y funcionalidades:

1. **Desarrollo de un Entorno de Aprendizaje por Refuerzo:** Implementación y validación de un entorno RL capaz de realizar la síntesis de circuitos Clifford, estableciendo una línea base funcional comparable al estado del arte (ej. enfoque PPO de IBM) [1].
2. **Implementación de un Módulo de Layout Multiobjetivo:** Creación de un componente de optimización basado en algoritmos evolutivos (como NSGA-II) [2] para determinar la asignación inicial de cúbits. Este módulo se diseñará para ser extensible, optimizando inicialmente métricas lógicas como profundidad y número de puertas CNOT, pero permitiendo la incorporación iterativa de nuevas funciones de coste (ej. tasas de error o decoherencia) según evolucione el proyecto.
3. **Integración del Pipeline Híbrido:** Orquestación del flujo de trabajo donde la solución del módulo multiobjetivo alimenta la entrada del agente de RL. Esta integración se realizará mediante iteraciones ("*sprints*") que aseguren la estabilidad del aprendizaje.
4. **Evaluación y Benchmarking:** Desarrollo de un sistema de pruebas para comparar el rendimiento del enfoque híbrido (MO+RL) frente a las heurísticas estándar, analizando la calidad del circuito final transpilado.

Listado de resultados que generará el TFG (aplicaciones, estudios, manuales, etc.)

Código Fuente: Repositorio con la implementación completa del pipeline híbrido (*Layout* multiobjetivo + Síntesis RL) en Python.

Memoria del TFG: Documento académico que incluye diseño, metodología y análisis de resultados experimentales.

Guía de Despliegue (Anexo): Instrucciones de instalación, requisitos y ejecución básica para garantizar la reproducibilidad, integradas en la memoria.

Datos Experimentales: Conjunto de circuitos de prueba (*benchmarks*), logs de entrenamiento y gráficas de rendimiento.

METODOLOGÍA:

Descripción de la metodología empleada en el desarrollo del TFG. Especificar cómo se va a desarrollar. Concretar si se trata de alguna metodología existente y, en caso contrario, describir y justificar adecuadamente los métodos que se aplicarán.

Se empleará una metodología ágil basada en una adaptación del marco de trabajo Scrum para un desarrollo individual. Esta elección se justifica por la naturaleza investigadora del proyecto, donde la incertidumbre inherente al entrenamiento de modelos de Aprendizaje por Refuerzo y algoritmos evolutivos requiere una gestión flexible de los cambios y una adaptación continua basada en los resultados experimentales. La estructura organizativa será la siguiente:

- **Roles:** Los tutores actuarán como *Product Owners*, siendo responsables de validar los avances y re-priorizar los objetivos del proyecto. Al tratarse de un desarrollo unipersonal, se unificarán las responsabilidades de Equipo de Desarrollo y Scrum Master, centralizando tanto la ejecución técnica como la gestión de las iteraciones en una única figura.
- **Ciclo de Vida (Mini-Sprints):** El cronograma se dividirá en iteraciones cortas o "mini-sprints". Cada sprint comenzará con una planificación de las tareas a abordar (extraídas de las fases de trabajo) y concluirá con una revisión con los tutores (*Sprint Review*).
- **Adaptabilidad:** Este enfoque permitirá inspeccionar el incremento del producto (código, gráficas de entrenamiento o borradores de memoria) al final de cada iteración. Basándose en la retroalimentación de los *Product Owners*, se podrán

integrar cambios en las métricas o algoritmos de forma dinámica sin comprometer la planificación global.

FASES DE TRABAJO:

Enumeración y breve descripción de las fases de trabajo en las que consistirá el TFG.

1. **Análisis y Diseño Arquitectónico:** Estudio bibliográfico sobre síntesis de circuitos mediante Aprendizaje por Refuerzo (RL) [2,5] y algoritmos evolutivos [3]. Incluye el diseño de la arquitectura híbrida y la selección de métricas de rendimiento (profundidad, CNOTs).
2. **Desarrollo del Entorno RL:** Comprende la configuración del entorno de simulación (**Gymnasium** [6] / **Qiskit** [4]), la implementación del agente de síntesis utilizando frameworks de RL robustos como **Stable Baselines3** [8] (basado en **PyTorch** [7]) y su validación inicial para establecer una línea base de rendimiento funcional.
3. **Implementación del Módulo Multiobjetivo:** Desarrollo y calibración del algoritmo de optimización de *layouts* (basado en librerías como **pymoo** [3]) y programación de las funciones de *fitness* extensibles para evaluar la calidad inicial de la asignación de cúbits.
4. **Integración y Validación Experimental:** Fusión de ambos módulos, alimentando el agente RL con *layouts* optimizados. Ejecución de *benchmarks* comparativos frente a heurísticas estándar y análisis estadístico de los resultados.
5. **Documentación y Entrega:** Redacción de la memoria técnica del TFG, elaboración de la guía de reproducibilidad (anexo) y preparación del repositorio de código final.

TEMPORIZACIÓN:

*La siguiente tabla deberá contener una fila por cada una de las fases enumeradas en la sección anterior. La temporización **no debe incluir las horas dedicadas a la preparación de la presentación y defensa del TFG**.*

*En caso de tratarse de un **trabajo en grupo**, se añadirá una columna HORAS por cada miembro del equipo. Debe especificarse claramente el número de horas dedicado por cada alumno/a y la suma de horas individual deberá ser también de 296.*

FASE	HORAS
	Eduardo González Bautista
1. Análisis y Diseño Arquitectónico	
1.1. Revisión bibliográfica: RL y transpilación cuántica	20
1.2. Estudio de algoritmos MO (NSGA-II/MOEA/D) y diseño	25
2. Prototipado del Entorno RL	
2.1. Configuración del entorno de simulación (Gymnasium/Qiskit)	20
2.2. Implementación del agente de síntesis (PPO/DQN)	25
2.3. Entrenamiento y validación del <i>baseline</i> [2]	20
3. Desarrollo del Módulo de Layout	
3.1. Implementación del algoritmo evolutivo de Layout	25
3.2. Desarrollo de funciones de fitness (Profundidad/CNOT)	25
3.3. Pruebas unitarias y ajuste de hiperparámetros MO	20
4. Integración y Experimentación	
4.1. Integración del pipeline híbrido (Layout -> RL)	25
4.2. Ejecución de benchmarks y recolección de métricas	25
4.3. Análisis de resultados y refinamiento (Iterativo)	15
5. Documentación y Cierre	
5.1. Redacción: Introducción, estado del arte y marco teórico	15
5.2. Redacción: Diseño técnico y metodología	20
5.3. Redacción: resultados, conclusiones y guía de uso	16
	296



UNIVERSIDAD
DE MÁLAGA





TECNOLOGÍAS EMPLEADAS:

Enumeración de las tecnologías utilizadas (lenguajes de programación, frameworks, sistemas gestores de bases de datos, etc.) en el desarrollo del TFG.

Python 3.9+

Qiskit SDK

PyTorch

Gymnasium

pymoo

Stable Baselines3

RECURSOS SOFTWARE Y HARDWARE:

Listado de dispositivos (placas de desarrollo, microcontroladores, procesadores, sensores, robots, etc.) o software (IDE, editores, etc.) empleados en el desarrollo del TFG.

Visual Studio Code / PyCharm / Jupyter Notebooks

Ordenador personal

Ordenador de la ETSII

Google Colab / Kaggle

IBM Quantum Platform

Listado de referencias (libros, páginas web, etc.)

[1] Nielsen, Michael A., and Isaac L. Chuang. **Quantum computation and quantum information**. Cambridge university press, 2010.

[2] D. Visconti et al., *AI-driven circuit synthesis with reinforcement learning* (arXiv:2405.13196); Mayo 2024, Accedido en 29/01/26. Disponible en: <https://arxiv.org/abs/2405.13196>

[3] J. Blank & K. Deb, *pymoo: Multi-objective Optimization in Python* (IEEE Access, vol. 8); 2020, Accedido en 29/01/26. Disponible en: <https://ieeexplore.ieee.org/document/9078759>

[4] IBM Quantum, *Qiskit AI Transpiler Passes Documentation*; 2024, Accedido en 29/01/26. Disponible en: <https://quantum.cloud.ibm.com/docs/en/guides/ai-transpiler-passes>

[5] Qiskit Contributors, *Qiskit IBM Transpiler Service* (GitHub Repository); 2024, Accedido en 29/01/26. Disponible en: <https://github.com/Qiskit/qiskit-ibm-transpiler>

[6] M. Towers et al., *Gymnasium: A Standard Interface for Reinforcement Learning Environments* (Farama Foundation); Marzo 2023, Accedido en 29/01/26. Disponible en: <https://gymnasium.farama.org/>

[7] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library* (NeurIPS 2019); 2019, Accedido en 29/01/26. Disponible en: <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>

[8] A. Raffin et al., *Stable Baselines3: Reliable Reinforcement Learning Implementations* (Journal of Machine Learning Research, vol. 22, no. 268); 2021, Accedido en 30/01/26. Disponible en: <http://jmlr.org/papers/v22/20-1364.html>

Málaga, 30 de enero de 2026

Firma tutor/tutora:

Firma cotutor/a:

Firma tutor/a coordinador/a:



UNIVERSIDAD
DE MÁLAGA

